

**Department of Electrical and Computer Engineering
Concordia University**

Communication Networks and Protocols - COEN 366

Winter 2026

Project: News Sharing System

Designed by: Ferhat Khendek

1. Introduction

The project consists of implementing in, Java, C++, C#, or Python, a news sharing system. The description of the protocol(s) to implement is given in the following section while the requirements are stated in Section 3.

2. The News Sharing System (NSS)

The News Sharing System (NSS) consists of servers and clients. The goal of the NSS is to allow for users to share information on subjects of interest, remain up to date on these subjects, and comment.

There are several servers and several clients (also known as users). For this project, let us assume two servers (two regions) only, and let us name them Server A and Server B. Both servers are serving at the same time, but different regions (different clients/users). Servers are known to the clients (users) in their respective regions (i.e. Server IP address, TCP socket number and UDP socket number are known to the clients). You can fix the TCP socket# of both servers to 10,000, and their UDP socket# to 20,000. Clients talk to the server of their respective region and vice versa. The servers know and talk to each other as described later.

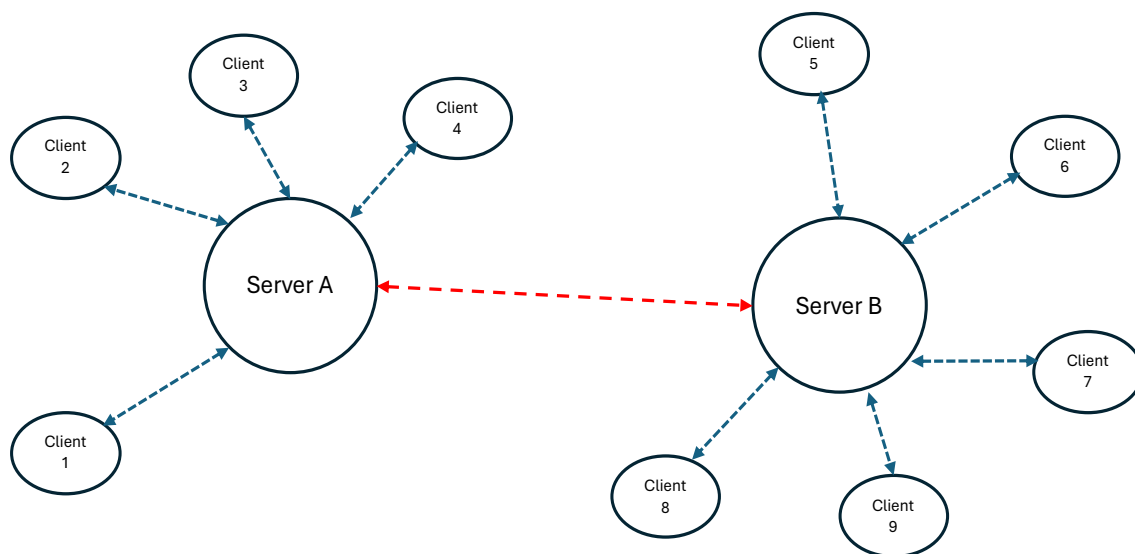


Figure 1. An illustration with 2 servers and 9 clients.

A user has a unique name, but where he/she can be reached (IP Address, etc.) may change over time. The users register to the server of the region, inform the servers of their subjects of interest, send and receive news from the server. A user can update his/her address (IP address, TCP socket# and UDP socket#), or subjects of interest anytime. A user will receive news about his/her subjects of interest only and can comment. A user can deregister; in this case he/she will not publish or receive news anymore.

More details about the protocol(s) are given below.

2.1. Registration (over TCP)

A user has to register with the server of the region before he/she can send subjects of interests or send/receive news.

For registering a user has to open a TCP connection to the region server and send his/her name (every user has a unique name), IP Address, TCP socket# and UDP socket# where it can be reached at using the following message:

REGISTER	RQ#	Name	IP Address	TCP Socket#	UDP Socket#
-----------------	-----	------	------------	-------------	-------------

Upon reception of this message the server, can accept or refuse the registration. For instance, the registration can be denied if the provided Name is already in use for instance, or the IP Address is wrong, TCP Socket# is wrong, etc., or the server cannot handle more users. If the registration is accepted the following message is sent to the user.

REGISTERED	RQ#
-------------------	-----

If the registration is denied, the server will send the following message and provide the reason.

REGISTER-DENIED	RQ#	Reason
------------------------	-----	--------

The RQ# is used to refer to which “REGISTER” message this confirmation or denial corresponds to.

Upon reception of REGISTER-DENIED, the user will retry or give up for a little while before retrying again depending on the reason.

When the registration is successful or the user gives up after a denial, the TCP connection is closed by the user. If the user wants to try to register again later, then he/she must open a new TCP connection.

Upon reception of a REGISTER message server may find that the client IP address is not in its region (cannot service this IP address). In this case the server refers the user to another server (in our case the second one) using the following message:

REFER	RQ#	IP Address of the other server
--------------	-----	--------------------------------

After receiving this message, the user closes the TCP connection with the first server and opens a new TCP connection with the second server to register with, following the normal registration procedure outlined in this sub-section.

2.2. Deregistration (over TCP)

For deregistering a user has to setup a TCP connection to the server and de-register by sending the following message:

DE-REGISTER	RQ#	Name
--------------------	-----	------

If the name is already registered, the server will remove the name and all the information related to this user.

In case the Name is not registered, for instance, the message is just ignored by the current server. No further action is taken.

The TCP connection is closed by the server.

2.3. Users updating their information (over TCP)

A registered user can always modify his/her IP address and/or socket numbers. A user can do so by opening a TCP connection to its server and send the following message:

UPDATE	RQ#	Name	IP Address	TCP Socket#	UDP Socket#
---------------	-----	------	------------	-------------	-------------

Upon reception of this message the current server can accept the update and reply to the user using the message:

UPDATE - CONFIRMED	RQ#	Name	IP Address	TCP Socket#	UDP Socket#
-------------------------------	-----	------	------------	-------------	-------------

In case of denial of update, because of errors such as “Name” does not exist, or IP address is incorrect, or Socket# is incorrect, the following message is sent to the user:

UPDATE-DENIED	RQ#	Reason
----------------------	-----	--------

The TCP connection is closed by the user if not needed anymore.

In case the server finds the user is registered with it, but the new IP address is not in its region (cannot service this IP address), it refers the user to another server (in our case the second one) as done during registration with the REFER message. The first server deregisters the user in question and closes the TCP connection. This user may then follow up with the other server for registration as done in sub-section 2.1.

2.4. Users updating their subjects of interest. (over TCP)

A registered user can update his/her subjects of interest by opening a TCP connection to the server it is registered with and send the following message:

SUBJECTS	RQ#	Name	List of subjects
-----------------	-----	------	------------------

Upon reception of this message the server can accept the update or reject it because of errors in the Name or in the list of subjects. You may assume a fixed list of subjects, for instance subjects related to computer engineering.

In the case of acceptance, the server sends the following message to the user:

SUBJECTS UPDATED	-	RQ#	Name	List of subjects
-----------------------------	---	-----	------	------------------

In case of denial of the update, the server sends the following message to the user.

SUBJECTS REJECTED	-	RQ#	Name	List of subjects
------------------------------	---	-----	------	------------------

The user may retry as many times as needed to achieve this update of subjects.
The TCP connection is closed by user when not needed anymore.

2.5. Users publishing and receiving messages on subjects of interest (over UDP)

From time-to-time users will publish messages related to their subjects of interest. These messages will be forwarded to all registered users (in all regions) who have interest in the subject of the message.

A user can publish a message on a subject of his/her interest using the following message:

PUBLISH	RQ#	Name	Subject	Title	Text
----------------	-----	------	---------	-------	------

This message is sent to its server only. The server checks the Name of the user, the Subject, and if the subject is in the list of subjects of interest for the user. If this is the case, the information is sent to all users who have this subject in their list of interest and who are registered with the server, using the following message:

MESSAGE	Name	Subject	Title	Text
----------------	------	---------	-------	------

Where Name is the name of the user who is publishing this information, the subject is the subject of the message, Title is the title of the news, and Text is the body of the message.

In the case of errors, for instance the Name does not exist (is not registered), the current server sends the following message to the user at the origin of the "PUBLISH" message:

PUBLISH-DENIED	RQ#	Reason
-----------------------	-----	--------

The server informs the other server (of the other region) of the new message by sending the following message:

FORWARD	Name	Subject	Title	Text
----------------	------	---------	-------	------

The server receiving this message will forward it to all its users who have this subject in their list of interests and who are registered with the server, using the following message:

MESSAGE	Name	Subject	Title	Text
----------------	------	---------	-------	------

2.6. Commenting messages (over UDP)

Any user receiving a message of interest can comment it using the following message:

PUBLISH - COMMENT	Name	Subject	Title	Text
------------------------------	------	---------	-------	------

The comment is sent to the server the message is received from. Name refers to the name of the user sending the comment, but the Subject and Title must refer to the original message it is commenting on. The text field contains the comment.

Upon reception of PUBLISH-COMMENT message, a server forwards it to all interested users registered with it using the message:

COMMENT	Name	Subject	Title	Text
----------------	------	---------	-------	------

The server also sends the same message to the other server (the second one) using the message:

PUBLISH - COMMENT	Name	Subject	Title	Text
------------------------------	------	---------	-------	------

The second server will forward the comment to all interested users registered with it, using the message:

COMMENT	Name	Subject	Title	Text
----------------	------	---------	-------	------

3. Requirements

Project should be done in teams of 3 students. You must send, by February 02, your team list including student names, ID numbers and **Concordia** email addresses to ferhat.khendek@concordia.ca.

Design and Implement the Users and Servers that follow the protocol(s) aforementioned. The coding of the protocol messages is part of your design, i.e. you have to come out with the appropriate coding of the messages. You can decide to use simple text message, etc.

The information stored in the servers must be persistent, i.e. if a server crashes and is restored it will recover all the information as before crashing.

Reporting: Servers and Clients must be reporting their communications with the entity to the users of the system using a log file or printing directly into the screen. In other words, during the demonstration, I would like to see the messages sent and received, progress and failures.

Assumptions/Error/Exception Handling

You should be aware that the description as it is, does not state everything. For instance, what happens if a User receives a response with a RQ# that does not correspond to any of its (pending) requests?

And more to be discussed during lectures ...

State and document clearly any assumption you make beyond the assumptions made by the instructor.

Extra: Note that in this project we do not require authentication of users. Extra marks will be given to students who add a real authentication scheme to the NSS system.

Also, a GUI is not required as long as we can run the clients and the servers and see what is going on with the messages. Extra marks will be given if you decide to build a GUI. This can help during the debugging of your code and during the demonstrations.

You must submit your final code by week 13 (through Moodle). You should also hand in your report (Through Moodle), by Week 13, where you document clearly your assumptions, design decisions, code, your testing, and experiments. You must also state clearly the contributions of every member of the group. **Note that every student must contribute technically (designing and coding the protocol) to the project.**

A demo will be held during Week 13 of this winter term. During the demo the members of the group must all be ready to answer questions regarding design, assumptions, coding, etc.

During the demo we will also go through the code itself as well as the report.

The project will be discussed further during lectures.

Keep in mind the rules/guidelines as stated in the course outline.